BÁO CÁO TỔNG QUAN HỆ THỐNG

BOCBANG BACKEND

Dịch vụ nhận dạng giọng nói thời gian thực trên board MediaTek G520

Phân tích nghiệp vụ · Kiến trúc · Hướng dẫn đấu nối · Cài đặt và vận hành · Đánh giá hiệu năng

Phiên bản tài liệu	1.0
Ngày lập	29/10/2026
Người lập	Dương Minh Hoàng
Mã nguồn	https://github.com/DuongMinhHoangHE191087/BocBang_Backend
Nhánh và phiên bản mã	main, commit 711fd68
Phiên bản hệ thống	0.0.1-SNAPSHOT (Spring Boot 4.1.2, Java 17)

Tài liệu phân biệt rõ số liệu đã đo, đã kiểm thử, theo mã nguồn và chưa test (xem mục 1.3). Nội dung chưa được đo trên board G520 được ghi "Chưa test", không thay bằng ước lượng.

MỤC LỤC

TÓM TẮT ĐIỀU HÀNH 4

1. GIỚI THIỆU 6

1.1 Mục đích tài liệu 6
1.2 Đối tượng đọc 6
1.3 Quy ước mức xác thực của số liệu 6
1.4 Nguồn mã nguồn và phiên bản 7

2. PHÂN TÍCH NGHIỆP VỤ (BA) 8

2.1 Bối cảnh và vấn đề cần giải quyết 8
2.2 Mục tiêu hệ thống 8
2.3 Các bên liên quan 9
2.4 Phạm vi 9
2.5 Yêu cầu chức năng 9
2.6 Yêu cầu phi chức năng 10
2.7 Tình huống sử dụng 11
2.8 Giả định và ràng buộc 11
2.9 Rủi ro nghiệp vụ và kỹ thuật 12

3. KIẾN TRÚC HỆ THỐNG 13

3.1 Tổng quan 13
3.2 Thành phần và quy mô 14
3.3 Luồng nhận dạng thời gian thực 14
3.4 Luồng xử lý tệp âm thanh dài 15
3.5 Luồng nhận dạng tệp ngắn đồng bộ 16
3.6 Mô hình đa luồng và kiểm soát tải 16
3.7 Lớp C++ JNI và bốn mô hình 17
3.8 Quyết định kiến trúc then chốt 17
3.9 Công nghệ sử dụng 18

4. HƯỚNG DẪN ĐẤU NỐI HỆ THỐNG KHÁC 19

4.1 Các điểm tích hợp 19
4.2 Giao thức WebSocket 19
4.3 Giao tiếp REST 20
4.4 Xác thực bằng khóa API 21
4.5 Hướng dẫn cho ứng dụng Android 21
4.6 Hướng dẫn cho hệ thống backend, web khác 22
4.7 Phiên bản và tương thích 22
4.8 Danh sách kiểm tra khi tích hợp 23

5. CÀI ĐẶT VÀ VẬN HÀNH 24

5.1 Lấy mã nguồn 24
5.2 Yêu cầu hệ thống 24
5.3 Chạy thử nhanh với engine giả (cấp A) 25
5.4 Chạy đầy đủ với mô hình thật trên PC (cấp B) 25

5.5 Triển khai lên board MediaTek G520 (cấp C) 26
5.6 Cấu hình quan trọng 26
5.7 Vận hành hằng ngày 28
5.8 Xử lý sự cố thường gặp 28
6. HIỆU NĂNG, SỨC CHỨA VÀ ĐÁNH GIÁ 29
6.1 Nguyên tắc và phạm vi đo 29
6.2 Môi trường đo 29
6.3 Kết quả kiểm thử tự động 29
6.4 Đo đồng thời với engine giả 30
6.5 Độ chính xác nhận dạng (WER) 31
6.6 Sức chứa kết nối trên board G520 32
6.7 Bộ nhớ 33
6.8 Giới hạn cứng và hành vi khi quá tải 34
6.9 Đánh giá tổng hợp 35
7. BẢO MẬT VÀ AN TOÀN VẬN HÀNH 36
7.1 Biện pháp hiện có 36
7.2 Khoảng trống và khuyến nghị 36
8. CHẤT LƯỢNG MÃ NGUỒN VÀ QUY TRÌNH 37
8.1 Quy mô mã nguồn 37
8.2 Quy trình phát triển 37
8.3 Điểm mạnh và điểm cần cải thiện 37
8.4 Việc còn nợ do nhóm phát triển ghi nhận 38
9. KẾT LUẬN VÀ ĐỀ XUẤT 39
9.1 Kết luận 39
9.2 Lộ trình đề xuất 39
9.3 Thông tin cần lãnh đạo và nghiệp vụ cung cấp 39
PHỤ LỤC A. THUẬT NGỮ 40
PHỤ LỤC B. BẢN ĐỒ KHO MÃ NGUỒN 41
PHỤ LỤC C. BIỂU MẪU GHI SỐ ĐO TRÊN BOARD G520 42

TÓM TẮT ĐIỀU HÀNH

BocBang Backend là máy chủ nhận dạng giọng nói (Speech-to-Text) viết bằng Spring Boot 4.1 và Java 17, chạy trên board MediaTek G520. Hệ thống nhận âm thanh từ ứng dụng Android hoặc hệ thống khác, nạp trực tiếp bốn mô hình AI định dạng GGML/GGUF vào bộ nhớ qua C++ JNI, rồi trả văn bản về theo hai cách: thời gian thực qua WebSocket, hoặc theo tệp qua REST (tệp ngắn xử lý đồng bộ, tệp dài chạy nền). Mã nguồn không có lời gọi mạng ra ngoài, nên toàn bộ quá trình nhận dạng diễn ra cục bộ.

Bảng 1. *Các chỉ số chính của hệ thống*

Chỉ số	Giá trị	Mức xác thực
Mã nguồn Java chạy thật	3.266 dòng trong 39 tệp	**Đã đo**
Mã kiểm thử Java	2.735 dòng trong 21 tệp (0,84 dòng kiểm thử trên mỗi dòng mã)	**Đã đo**
Mã C++ JNI	639 dòng, biên dịch thành 3 thư viện .so	**Đã đo**
Kiểm thử tự động	117 test: 115 đạt, 2 bỏ qua, 0 lỗi; chạy mất 15,6 giây	**Đã đo**
Tệp đóng gói	Một tệp JAR 25,7 MB	**Đã đo**
Khởi động (engine giả, máy phát triển)	4,0 giây; sẵn sàng nhận kết nối sau 5,5 giây	**Đã đo**
Bộ nhớ tiến trình Java (engine giả, giới hạn heap 512 MB)	181 đến 215 MB	**Đã đo**
Kiểm soát tải (engine giả, 40 kết nối vào máy chủ giới hạn 28)	Nhận 28, từ chối đúng 12 kết nối	**Đã đo**
Độ trễ lớp điều phối (engine giả, đến 28 kết nối)	0 ms ở p50, p95, p99; tối đa 1 ms	**Đã đo**
Số luồng thời gian thực tối đa trên board G520	7 theo cấu hình mặc định (khi JVM thấy 8 CPU)	**Theo mã nguồn**
Độ chính xác nhận dạng (tỷ lệ lỗi từ, WER)	Chưa có số đo	**Chưa test**
Tốc độ và sức chứa thật với mô hình thật trên board	Chưa có số đo	**Chưa test**

Đánh giá nhanh

- **Điểm tốt:** phần điều phối đa luồng, giao thức, bảo mật và xử lý lỗi có kiểm thử dày (117 test, không lỗi). Giới hạn tải hoạt động đúng khi đo với engine giả. Có sẵn health probe, metrics, tài liệu tiếng Việt và ghi chép 14 quyết định kiến trúc.
- **Chưa chứng minh được:** chất lượng nhận dạng và sức chứa với mô hình thật trên board. Môi trường lập báo cáo chưa có thư viện .so và tệp mô hình, nên hai nội dung này ghi “Chưa test”, không ước lượng thay số đo.

- **Điều cần lưu ý cho cam kết:** mọi lời gọi mô hình chạy lần lượt từng cái một (CpuExecutionGate), nên thêm kết nối không làm tăng thông lượng. Con số 7 luồng là mức cấu hình thận trọng, chưa phải sức chứa đã kiểm chứng.

Khuyến nghị

1. Đo sức chứa trên board G520 bằng công cụ thử tải có sẵn với tiếng nói thật, rồi đặt giới hạn luồng theo số đo (mục 6.6).
2. Đo độ chính xác (WER) trên bộ tệp họp tiếng Việt do nghiệp vụ cung cấp (mục 6.5).
3. Đặt máy chủ sau proxy HTTPS và bật khóa API trước khi mở ra ngoài mạng nội bộ (mục 7).

1. GIỚI THIỆU

1.1 Mục đích tài liệu

Tài liệu cung cấp cho lãnh đạo và nhóm kỹ thuật bức tranh đầy đủ về BocBang Backend: hệ thống phục vụ nghiệp vụ gì, được thiết kế ra sao, các hệ thống khác đấu nối như thế nào, cách cài đặt và chạy, kết quả đo đạc hiện có, các giới hạn đã biết và việc cần làm tiếp theo. Mã nguồn được công bố để người xem có thể tự đọc và chạy lại.

1.2 Đối tượng đọc

Bảng 2. *Gợi ý mục cần đọc theo vai trò*

Vai trò	Nên đọc
Lãnh đạo, chủ sản phẩm	Tóm tắt điều hành; chương 6 (đánh giá); chương 9 (kết luận và đề xuất)
Nhà phân tích nghiệp vụ	Chương 2 (phân tích nghiệp vụ); mục 4.1
Kiến trúc sư, lập trình viên backend	Chương 3 (kiến trúc); chương 8 (chất lượng mã nguồn)
Lập trình viên Android, đội tích hợp	Chương 4 (hướng dẫn đấu nối)
Vận hành, triển khai	Chương 5 (cài đặt, vận hành); mục 6.6 đến 6.8; chương 7 (bảo mật)

1.3 Quy ước mức xác thực của số liệu

Mỗi con số hoặc nhận định kỹ thuật trong tài liệu gắn một nhãn cho biết nó đến từ đâu, để người đọc không nhầm số đo thật với suy luận.

Bảng 3. *Năm mức xác thực*

Nhãn	Ý nghĩa
Đã đo	Số liệu đo thực tế trong phiên lập báo cáo ngày 29/10/2026, trên máy phát triển (không phải board G520).
Đã kiểm thử	Có bài kiểm thử tự động đạt trong phiên lập báo cáo.
Theo mã nguồn	Đọc trực tiếp từ mã nguồn, cấu hình hoặc tài liệu trong kho; chưa chạy riêng để kiểm chứng.
Chưa test	Chưa có số đo hoặc bài kiểm chứng. Không được dùng như thông tin đã xác nhận.
Cần bổ sung	Thông tin nằm ngoài mã nguồn (mục tiêu kinh doanh, chỉ tiêu chất lượng, cấu hình phần cứng); cần người phụ trách cung cấp.

1.4 Nguồn mã nguồn và phiên bản

Bảng 4. *Thông tin kho mã nguồn*

Hạng mục	Giá trị
Địa chỉ kho	https://github.com/DuongMinhHoangHE191087/BocBang_Backend (công khai)
Nhánh mặc định	main
Phiên bản mã dùng cho báo cáo	Commit 711fd68 ngày 29/10/2026; phiên bản ứng dụng 0.0.1-SNAPSHOT
Lịch sử	38 commit; 2 lập trình viên và Dependabot (tự động nâng phụ thuộc)
Kết quả CI mới nhất trên nhánh main	Thành công (29/10/2026)
Tài liệu có sẵn trong kho	README.md: cách dùng, API, cấu hình, triển khai docs/quyet-dinh-kien-truc.md: 14 quyết định kiến trúc và việc còn nợ .claude/skills/stt-backend/SKILL.md và CLAUDE.md: tóm tắt kiến trúc, quy ước cho người viết mã

> **Về tài liệu phân tích nghiệp vụ (BA)**
> Kho mã chưa có tài liệu BA riêng. Chương 2 được lập lại từ README, ghi chép quyết định kiến trúc và mã nguồn. Các thông tin nghiệp vụ không có trong mã (mục tiêu kinh doanh, người dùng mục tiêu, chỉ tiêu chất lượng, ngân sách) được đánh dấu **Cần bổ sung**.

2. PHÂN TÍCH NGHIỆP VỤ (BA)

2.1 Bối cảnh và vấn đề cần giải quyết

Các ứng dụng ghi chép cuộc họp và ghi âm cần chuyển lời nói thành văn bản. Dùng dịch vụ đám mây làm dữ liệu giọng nói rời khỏi mạng nội bộ và phụ thuộc đường truyền. Board MediaTek G520 có CPU ARM đủ sức chạy các mô hình nhận dạng đã lượng tử hóa, vì vậy dự án gom các mô hình đó vào một máy chủ dùng chung cho nhiều ứng dụng trong mạng nội bộ. Bối cảnh "ghi chép cuộc họp" được suy ra từ tên thư mục mô hình trong README; bối cảnh kinh doanh chính thức: **Cần bổ sung**.

Hệ thống giải quyết bốn vấn đề:

- Nhận dạng giọng nói tiếng Việt và tiếng Anh không cần Internet, trên phần cứng nhỏ.
- Phục vụ nhiều ứng dụng và người dùng cùng lúc mà không làm quá tải CPU và RAM vốn hạn chế.
- Xử lý được cả cuộc họp dài (từ 5 phút đến 1 giờ) lẫn câu nói ngắn theo thời gian thực.
- Cho phép thay hoặc thêm mô hình mà không phải sửa phần giao tiếp với client.

2.2 Mục tiêu hệ thống

Bảng 5. *Mục tiêu và cách hệ thống đáp ứng*

Mã	Mục tiêu	Cách hệ thống đáp ứng
G1	Chuyển lời nói thành văn bản cục bộ	Bốn mô hình GGML/GGUF nạp trực tiếp qua JNI, không gọi tiến trình hay dịch vụ ngoài
G2	Phục vụ đồng thời nhiều client	Giới hạn số luồng, hàng đợi riêng cho mỗi kết nối, bộ thread cố định dùng chung
G3	Bảo vệ máy khỏi quá tải	Từ chối sớm khi đầy, hàng đợi có giới hạn, hủy tức thì phép tính native
G4	Dễ tích hợp	WebSocket và REST phiên bản v1, lỗi theo RFC 9457, tài liệu giao thức đầy đủ
G5	Dễ vận hành	Health liveness và readiness, metrics Prometheus, tắt máy chủ êm, tự dọn tệp tạm
G6	Dễ mở rộng mô hình	Giao diện SpeechToTextEngine kèm bộ kiểm thử hợp đồng cho engine mới

2.3 Các bên liên quan

Bảng 6. *Danh sách bên liên quan (suy ra từ tài liệu kỹ thuật)*

Bên liên quan	Vai trò	Quan tâm chính
Người dùng cuối	Nói vào micro hoặc tải tệp họp, đọc văn bản	Chữ đúng, ra nhanh
Nhóm ứng dụng Android	Viết client gửi PCM qua WebSocket	Giao thức ổn định, mã lỗi rõ ràng
Nhóm AI, mô hình	Cung cấp và thay mô hình	Cắm engine mới mà không sửa phần giao tiếp
Nhóm phát triển backend	Duy trì mã nguồn	Kiểm thử, CI, ghi chép quyết định
Vận hành, triển khai board	Cài đặt, cấp khóa API, giám sát	Script đẩy lên board, health, metrics
Lãnh đạo, chủ sản phẩm	Quyết định đầu tư và phát hành	Số liệu hiệu năng, rủi ro (tên, KPI cụ thể: **Cần bổ sung**)

2.4 Phạm vi

Bảng 7. *Trong và ngoài phạm vi hiện tại*

Trong phạm vi	Ngoài phạm vi (chưa có trong mã)
Nhận dạng thời gian thực qua WebSocket (PCM 16 kHz, 16-bit, mono) Nhận dạng đồng bộ tệp WAV ngắn (tối đa 60 giây) Xử lý nền tệp dài (mp3, m4a, ogg, flac, wav) qua ffmpeg Sửa dấu câu, viết hoa bằng mô hình ngôn ngữ Khóa API tùy chọn, health, metrics Web demo và engine giả để thử	Chạy Spring Boot bên trong APK Android (app chỉ là client) Cơ sở dữ liệu và lưu trữ lâu dài (kết quả job giữ tối đa 24 giờ) Mã hóa đường truyền (cần proxy HTTPS) Huấn luyện hoặc tinh chỉnh mô hình Giao diện quản trị, tách người nói, dịch thuật

2.5 Yêu cầu chức năng

Bảng 8. *Danh sách yêu cầu chức năng và mức kiểm chứng*

Mã	Yêu cầu	Thành phần đáp ứng	Mức xác thực
FR-01	Nhận PCM 16 kHz mono qua WebSocket /ws/stt và trả văn bản dạng JSON	SpeechWebSocketHandler	**Đã kiểm thử**
FR-02	Chốt câu bằng lệnh end; mỗi lệnh nhận đúng một kết quả	SessionPipeline (20 test)	**Đã kiểm thử**
FR-03	Nhận dạng đồng bộ một tệp WAV ngắn qua POST /transcriptions	TranscriptionService	**Đã kiểm thử**
FR-04	Nhận tệp dài, xử lý nền, xem tiến độ, hủy job	TranscriptionJobService (12 test)	**Đã kiểm thử**
FR-05	Đổi mọi định dạng âm thanh sang WAV chuẩn bằng ffmpeg	FfmpegAudioConverter	**Theo mã nguồn**
FR-06	Tách đoạn có tiếng nói bằng Silero VAD	JniVoiceActivityDetector	**Chưa test**
FR-07	Nhận dạng bằng Qwen3-ASR (đa ngôn ngữ, tiếng Việt) hoặc Whisper small EN	JniSpeechEngine	**Chưa test**

Mã	Yêu cầu	Thành phần đáp ứng	Mức xác thực
FR-08	Sửa dấu câu và lỗi nhận dạng bằng Qwen3-4B	JniTranscriptRefiner	Chưa test
FR-09	Giới hạn số luồng đồng thời, từ chối sớm khi đầy	SessionLimiter	Đã đo
FR-10	Xác thực bằng khóa API (Bearer hoặc X-Api-Key; tham số apiKey cho WebSocket)	ApiKeyFilter (22 test bảo mật)	Đã kiểm thử
FR-11	Báo trạng thái, sức chứa, health probe và metrics	SttController, Actuator	Đã kiểm thử
FR-12	Trả lỗi theo RFC 9457, không lộ đường dẫn hay lỗi native	ApiExceptionHandler	Đã kiểm thử
FR-13	Thêm engine nhận dạng mới mà không sửa phần giao tiếp	SpeechToTextEngine, bộ test hợp đồng	Đã kiểm thử
FR-14	Web demo: micro, tệp âm thanh, nhiều kết nối song song	Thư mục static	Theo mã nguồn
FR-15	Hủy job hoặc quá thời gian thì dừng ngay phép tính native	CpuExecutionGate	Theo mã nguồn

2.6 Yêu cầu phi chức năng

Bảng 9. *Danh sách yêu cầu phi chức năng và mức kiểm chứng*

Mã	Nhóm	Yêu cầu và giới hạn	Mức xác thực
NFR-01	Hiệu năng	Thread web không chạy mô hình; tổng CPU dùng cho nhận dạng bị chặn bởi số worker	Đã kiểm thử
NFR-02	Hiệu năng	Độ trễ lớp điều phối bằng 0 ms đến 28 kết nối (engine giả)	Đã đo
NFR-03	Sức chứa	Mặc định 7 luồng thời gian thực trên board 8 nhân; sức chịu tải thật với mô hình thật	Theo mã nguồn / Chưa test
NFR-04	Bộ nhớ	Heap Java tối đa 512 MB trên board; mô hình ASR khoảng 1,3 GB, refiner khoảng 2,5 GB, không nạp cùng lúc	Theo mã nguồn
NFR-05	Tin cậy	Từ chối đầu vào xấu giữ kết nối; lỗi native đóng phiên; 50 lần từ chối liên tiếp thì đóng	Đã kiểm thử
NFR-06	Bảo mật	Khóa API tùy chọn; lỗi không lộ đường dẫn; chặn vượt bộ lọc bằng path parameter	Đã kiểm thử
NFR-07	Quan sát	9 metric riêng (tiền tố stt) cùng probe liveness và readiness	Theo mã nguồn
NFR-08	Vận hành	Tắt êm (chờ tối đa 20 giây); dọn thư mục job khi tắt và khi khởi động	Theo mã nguồn
NFR-09	Tương thích	Java 17; Linux ARM64, Android, PC x86_64; client chỉ cần WebSocket và HTTP	Theo mã nguồn
NFR-10	Bảo trì	CI chạy toàn bộ test mỗi pull request; Dependabot nâng phụ thuộc hằng tuần	Theo mã nguồn

Mã	Nhóm	Yêu cầu và giới hạn	Mức xác thực
NFR-11	Chất lượng	Chỉ tiêu độ chính xác (WER) tối thiểu	**Cần bổ sung**
NFR-12	Sẵn sàng	Chỉ tiêu thời gian hoạt động (SLA) và độ trễ chấp nhận được	**Cần bổ sung**

2.7 Tình huống sử dụng

Bảng 10. *Các tình huống sử dụng chính*

Mã	Tình huống	Tác nhân	Luồng chính	Kết quả
UC-01	Ghi âm trực tiếp từ ứng dụng	Ứng dụng Android	Mở WebSocket; nhận ready; gửi PCM mỗi 100 ms; gửi end khi ngừng nói; nhận final	Văn bản từng câu
UC-02	Chuyển biên bản cuộc họp dài	Hệ thống khác	Tải tệp lên /jobs; nhận jobId; hỏi tiến độ; lấy văn bản khi xong	Văn bản và các đoạn
UC-03	Nhận dạng nhanh tệp ngắn	Hệ thống khác	Gửi WAV tối đa 60 giây tới / transcriptions	Văn bản (JSON)
UC-04	Giám sát và cân bằng tải	Vận hành, load balancer	Hỏi readiness; Prometheus lấy metrics	503 BUSY khi đầy
UC-05	Thêm mô hình mới	Nhóm AI	Viết engine mới, đặt biến STT_ENGINE	Engine chạy được
UC-06	Hủy job đang chạy	Client	Gửi DELETE theo jobId	Dừng tính toán, xóa tệp tạm

2.8 Giả định và ràng buộc

- Âm thanh thời gian thực phải là 16 kHz, 16-bit, mono; tần số khác bị từ chối. Tệp dài được ffmpeg tự chuyển về chuẩn này.
- Một phép tính native chạy tại một thời điểm, nên thông lượng không tăng theo số thread.
- Engine jni chỉ trả văn bản sau lệnh end; không có kết quả tạm (partial). Engine giả mới có partial.
- RAM của board là giới hạn thật; tổng RAM của board: **Cần bổ sung**.
- Máy chủ đặt trong mạng tin cậy; muốn mở ra ngoài phải qua proxy HTTPS.
- Tệp mô hình và thư viện .so không nằm trong kho mã (bị loại khỏi git); người chạy phải tự có.

2.9 Rủi ro nghiệp vụ và kỹ thuật

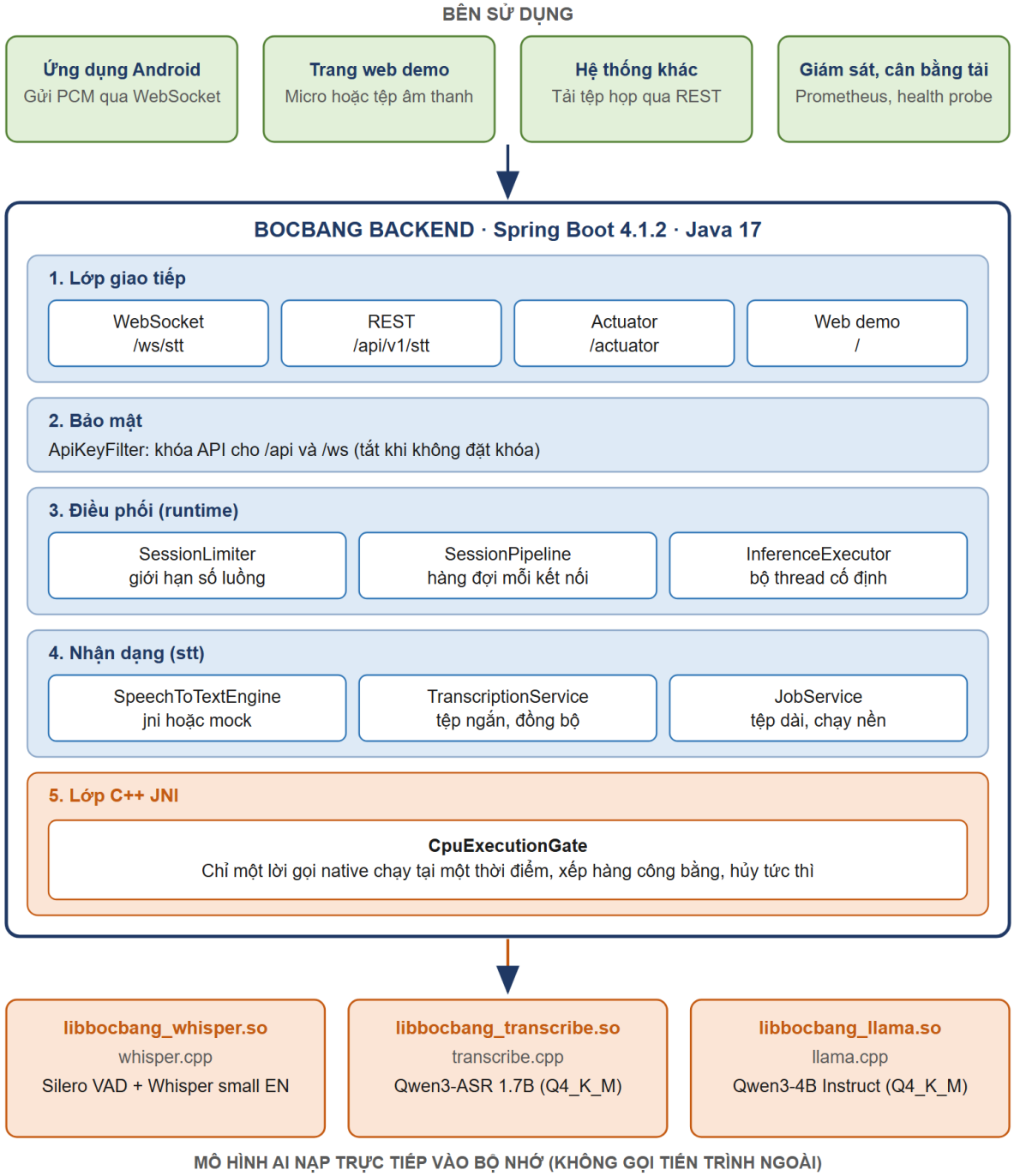
Bảng 11. *Rủi ro chính*

Mã	Rủi ro	Mức độ	Biện pháp
R1	Chưa biết sức chứa thật của board với mô hình thật; cam kết số người dùng có thể sai	Cao	Đo theo quy trình mục 6.6 trước khi cam kết
R2	Chưa đo độ chính xác tiếng Việt trên dữ liệu thực tế	Cao	Đo WER trên bộ tệp thật (mục 6.5)
R3	Job tệp dài chiếm cổng native, làm chậm người đang nói trực tiếp	Trung bình	Đo ảnh hưởng; tách giờ chạy job hoặc giới hạn job
R4	Sau khi refine, mô hình ASR phải nạp lại, câu kế tiếp chờ nhiều giây	Trung bình	Đo; cân nhắc tắt unload-before-refine nếu RAM đủ
R5	Mở dịch vụ khi chưa đặt khóa API và HTTPS	Trung bình	Bật khóa API, đặt sau proxy HTTPS (chương 7)
R6	Người ngoài khó dựng lại môi trường (mô hình, mã nguồn C++ ngoài kho, đường dẫn cứng trong CMake)	Trung bình	Tài liệu hóa nguồn tải và ghim phiên bản (mục 8.3)
R7	Phụ thuộc ba thư viện C++ bên thứ ba chưa được ghim phiên bản trong kho	Trung bình	Dùng submodule hoặc ghi rõ phiên bản

3. KIẾN TRÚC HỆ THỐNG

3.1 Tổng quan

Hệ thống gồm năm lớp xếp chồng: giao tiếp, bảo mật, điều phối, nhận dạng và lớp C++ JNI. Client gọi vào lớp giao tiếp; lớp điều phối bảo đảm tổng tải không vượt mức cho phép; lớp nhận dạng che giấu mô hình cụ thể sau một giao diện chung; lớp JNI nạp mô hình vào bộ nhớ và chạy từng phép tính một.



Hình 1. *Bối cảnh và các lớp của hệ thống*

3.2 Thành phần và quy mô

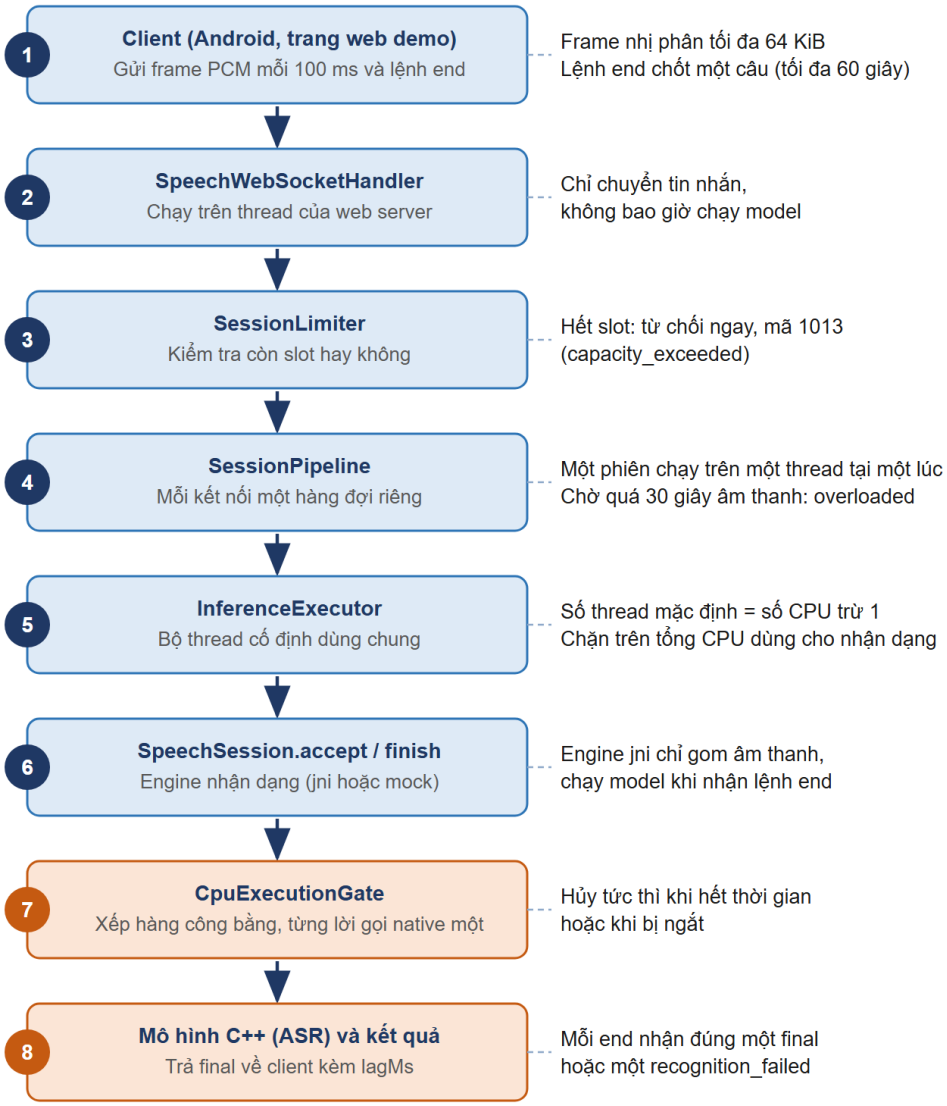
Bảng 12. *Các gói mã nguồn chính (số liệu đếm từ mã)*

Gói	Trách nhiệm	Tệp	Dòng
api	REST /api/v1/stt, bí danh /api/stt cũ, xử lý lỗi RFC 9457	3	232
websocket	Cổng WebSocket /ws/stt; chỉ chuyển tin nhắn, không chạy mô hình	2	307
runtime	InferenceExecutor, SessionPipeline, SessionLimiter, cấu hình, health indicator	7	592
security	ApiKeyFilter: khóa API cho /api và /ws	2	146
stt	Giao diện engine, phiên nhận dạng, dịch vụ tệp ngắn và job tệp dài, Wav, cấu hình	13	1.032
stt/jni	Engine JNI, CpuExecutionGate, VAD, refiner, ffmpeg, nạp thư viện	10	804
stt/mock	Engine giả để demo và thử tải	1	139
Điểm vào ứng dụng	BocBangBackendApplication	1	14
Tổng		39	3.266

3.3 Luồng nhận dạng thời gian thực

Mỗi kết nối WebSocket đi qua cùng một chuỗi xử lý, minh họa ở Hình 2. Thread của web server chỉ nhận tin và đặt vào hàng đợi; việc chạy mô hình luôn diễn ra trên bộ thread riêng. Các nguyên tắc bắt buộc của luồng này:

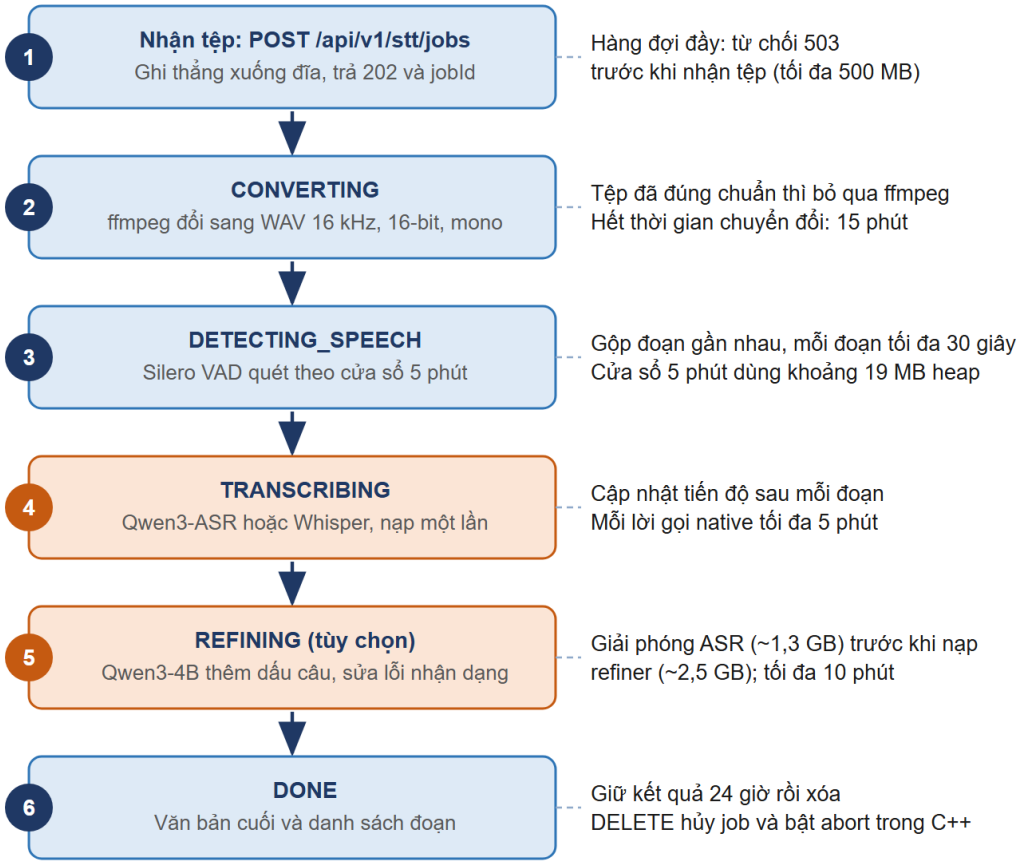
- Không chạy mô hình trên thread web và không tạo thread riêng cho engine.
- Slot của luồng chỉ được trả khi recognizer đã đóng thật, không phải lúc ngắt kết nối, để số recognizer sống không vượt giới hạn lúc quá tải.
- Lỗi “từ chối một đầu vào” (âm thanh không hợp lệ, không ra văn bản cho một câu) giữ kết nối và báo cho client; mọi lỗi khác, kể cả lỗi từ native, đóng phiên bằng engine_failure.
- Gửi tin từ thread khác chỉ qua ConcurrentWebSocketSessionDecorator.



Hình 2. *Đường đi của một phiên nhận dạng thời gian thực*

3.4 Luồng xử lý tệp âm thanh dài

Tệp 5 phút đến 1 giờ không được nạp cả vào bộ nhớ Java: tệp ghi thẳng xuống đĩa và được đọc theo từng cửa sổ. Mỗi job đi qua các giai đoạn sau; client xem được giai đoạn và tiến độ ở bất kỳ lúc nào.



Hình 3. *Quy trình xử lý tệp âm thanh dài*

3.5 Luồng nhận dạng tệp ngắn đồng bộ

Yêu cầu POST /transcriptions nhận một tệp WAV 16 kHz, 16-bit, mono dài tối đa 60 giây và trả văn bản ngay trong phản hồi. Luồng này dùng chung số slot và bộ thread với WebSocket, nên được tính vào giới hạn đồng thời. Tệp dài hơn 60 giây bị từ chối và phải dùng /jobs. Lý do: luồng này giữ cả tệp trong RAM, trong khi giới hạn tải lên của job là 500 MB.

3.6 Mô hình đa luồng và kiểm soát tải

Bảng 13. *Các cơ chế bảo vệ tải*

Cơ chế	Tác dụng	Khi vượt giới hạn
SessionLimiter	Giới hạn số luồng đồng thời (WebSocket và REST đồng bộ)	Từ chối ngay: WebSocket mã 1013, REST 503
SessionPipeline	Mỗi kết nối một hàng đợi; một phiên chạy trên một thread tại một lúc; gộp âm thanh tối đa 500 ms mỗi lần gọi; xếp lại cuối hàng sau mỗi lượt để công bằng	Chờ quá 30 giây âm thanh: lỗi overloaded, đóng 1013
InferenceExecutor	Bộ thread cố định dùng chung (mặc định số CPU trừ 1)	Tổng CPU nhận dạng bị chặn trên
CpuExecutionGate	Khóa công bằng cho mọi lời gọi native (VAD, ASR, refiner), chạy từng cái một	Quá 5 phút (ASR) hoặc 10 phút (refiner): hủy phép tính

Cơ chế	Tác dụng	Khi vượt giới hạn
Hàng đợi job	Một job chạy tại một lúc, tối đa 20 job chờ	Hàng đợi đầy: 503, trước khi nhận tệp
Gửi tin ra client	Giới hạn 10 giây hoặc 512 KiB chờ gửi	Ngắt client nhận chậm
Ngắt kết nối nhàn rỗi	Không gửi gì trong 60 giây	Đóng với mã 1001

> **Hệ quả cần nhớ về thông lượng**
> Vì CpuExecutionGate chạy từng lời gọi native một, việc tăng số worker không làm tăng thông lượng nhận dạng. Thêm worker chỉ giúp các phiên khác vẫn nhận kịp âm thanh trong lúc một phiên đang chờ mô hình. Sức chứa thật vì thế phụ thuộc tốc độ của một lần chạy mô hình trên board (**Chưa test**).

3.7 Lớp C++ JNI và bốn mô hình

Mỗi thư viện C++ đi kèm phiên bản ggml riêng, nên mô-đun native biên dịch thành ba tệp .so độc lập. Mỗi tệp ẩn toàn bộ ký hiệu ggml nội bộ (tệp exports.map và tùy chọn ẩn ký hiệu) để ba thư viện nạp chung vào một JVM mà không xung đột. Bài kiểm thử xác nhận điều này hiện bị bỏ qua vì môi trường lập báo cáo thiếu .so và mô hình, nên chưa được chạy: **Chưa test**.

Bảng 14. *Ba thư viện JNI và bốn mô hình*

Thư viện	Engine C++	Mô hình	Vai trò, bộ nhớ
libbocbang_whisper.so	whisper.cpp	ggml-silero-vad.bin whisper-small-en.bin	Tách đoạn tiếng nói; nhận dạng tiếng Anh RAM: **Cần bổ sung**
libbocbang_transcribe.so	transcribe.cpp	qwen3-asr-1.7b-q4_k_m.gguf	Nhận dạng đa ngôn ngữ, tiếng Việt (mặc định) RAM: khoảng 1,3 GB
libbocbang_llama.so	llama.cpp	qwen3-4b-instruct-2507-q4_k_m.gguf	Refiner: thêm dấu câu, viết hoa, sửa lỗi RAM: khoảng 2,5 GB

Cấu hình biên dịch cho board nhắm tới ARMv8.2-A kèm tập lệnh dotprod và fp16 (Cortex-A78 và Cortex-A55 của MediaTek Genio 520/720). Số thread tính toán bên trong mỗi lần chạy mô hình mặc định là 4; ghi chú trong cấu hình cho rằng 4 là tối ưu cho G520, nhưng kho không kèm số đo chứng minh (**Chưa test**).

3.8 Quyết định kiến trúc then chốt

Nhóm phát triển ghi lại 14 quyết định trong tệp docs/quyet-dinh-kien-truc.md. Bảng dưới trích tám quyết định ảnh hưởng nhiều nhất đến người dùng hệ thống.

Bảng 15. *Tám quyết định kiến trúc quan trọng*

Số	Quyết định	Lý do	Hệ quả
1	Tách hai bộ cấu hình cùng tiền tố stt	Tránh vỡ test của phần JNI và xung đột khi hai bên cùng sửa	Mỗi bộ bỏ qua khóa của bộ kia
2	Mọi lần chạy mô hình đi qua gói runtime	Thread web không chạy mô hình; chặn tổng CPU	Thêm worker không tăng thông lượng với engine jni
3	Lỗi engine chia hai loại	Một câu lỗi không nên đuổi cả người dùng	50 lần từ chối liên tiếp thì đóng phiên
4	Trả slot khi recognizer đã đóng thật	Lúc quá tải, lệnh đóng xếp sau các tác vụ khác	Không trả slot trong handler
5	Âm thanh chờ tối đa 30 giây	Engine jni chỉ trả sau end, có thể mất nhiều giây	Cần xác nhận lại bằng số đo trên board
6	Trạng thái BUSY khi đầy slot	Load balancer ngừng gửi người mới, monitor không coi là hỏng	Health 200, readiness 503
7	Giữ /api/stt cũ làm bí danh deprecated	Client cũ đang dùng	Kèm header Deprecation; chưa có ngày gỡ
8	Không gửi lỗi nội bộ cho client	Từng lộ đường dẫn tuyệt đối của máy chủ	Chi tiết chỉ vào log, client nhận mô tả chung

3.9 Công nghệ sử dụng

Bảng 16. *Danh mục công nghệ*

Thành phần	Công nghệ	Phiên bản
Ngôn ngữ	Java	17
Khung ứng dụng	Spring Boot: Web MVC, WebSocket, Actuator	4.1.2
Máy chủ web	Tomcat nhúng của Spring Boot	Theo Spring Boot
Đóng gói và build	Gradle (wrapper)	9.8.2
Giám sát	Micrometer, Prometheus	Theo Spring Boot
Mã native	C++17, CMake, Ninja, Android NDK	CMake từ 3.22
Thư viện mô hình	whisper.cpp, transcribe.cpp, llama.cpp (GGML/GGUF)	Chưa ghim trong kho
Chuyển đổi âm thanh	ffmpeg	Bản tĩnh ARM64 do script tải
Kiểm thử	JUnit 5 qua spring-boot-starter-test	Theo Spring Boot
CI	GitHub Actions (Ubuntu, JDK 17 Temurin)	Tự động mỗi pull request

4. HƯỚNG DẪN ĐẤU NỐI HỆ THỐNG KHÁC

4.1 Các điểm tích hợp

Hệ thống khác đấu nối qua hai kênh: WebSocket cho âm thanh thời gian thực và REST cho tệp. Ngoài ra có các đường dẫn trạng thái và giám sát. Cổng mặc định cấu hình là 8099 (biến SERVER_PORT).

Bảng 17. *Danh sách điểm tích hợp*

Mục đích	Giao thức	Địa chỉ	Dùng khi
Nhận dạng thời gian thực	WebSocket	ws://<máy chủ>:8099/ws/stt?sampleRate=16000	Ứng dụng ghi âm trực tiếp
Tạo job tệp dài	POST (multipart)	/api/v1/stt/jobs, tham số model và refine	Biên bản cuộc họp
Xem tiến độ, kết quả job	GET	/api/v1/stt/jobs/{jobId}	Sau khi tạo job
Hủy job	DELETE	/api/v1/stt/jobs/{jobId}	Người dùng hủy
Nhận dạng tệp ngắn	POST (multipart)	/api/v1/stt/transcriptions	WAV tối đa 60 giây
Trạng thái và sức chứa	GET	/api/v1/stt/status	Kiểm tra sẵn sàng
Probe sống và sẵn sàng	GET	/actuator/health/liveness và /actuator/health/readiness	Load balancer, giám sát
Metrics	GET	/actuator/prometheus và /actuator/metrics	Prometheus

4.2 Giao thức WebSocket

Quy trình một phiên

1. Mở kết nối tới địa chỉ WebSocket. Tham số sampleRate tùy chọn, mặc định 16000; tần số khác bị từ chối.
2. Nhận tin ready (tần số, định dạng pcm_s16le, 1 kênh, kích thước frame tối đa 65.536 byte). Nếu nhận lỗi capacity_exceeded thì chờ vài giây và thử lại.
3. Gửi các frame nhị phân PCM 16-bit little-endian, mono, 16 kHz; số byte chẵn; nên 100 ms mỗi frame (3.200 byte); tối đa 64 KiB mỗi frame.
4. Khi người nói ngừng (khoảng 0,8 giây), gửi tin văn bản end để chốt câu.
5. Nhận final của câu. Mỗi lệnh end nhận đúng một final hoặc một recognition_failed. Nhiều lệnh end liên tiếp mà không có âm thanh xen giữa chỉ nhận một final.
6. Kết thúc bằng cách đóng kết nối (mã 1000).

Tin nhắn từ máy chủ

Bảng 18. *Các loại tin nhắn máy chủ gửi về*

Loại	Trường chính	Ý nghĩa
ready	sampleRate, encoding, channels, maxFrameBytes	Gửi một lần khi kết nối thành công
partial	text, lagMs	Kết quả tạm; chỉ engine hỗ trợ streaming (engine giả); engine jni không gửi
final	text, lagMs	Kết quả của một câu sau lệnh end; lagMs là thời gian âm thanh mới nhất chờ trong máy chủ
error	code, message	Báo lỗi theo bảng mã lỗi bên dưới

Mã lỗi WebSocket và cách xử lý phía client

Bảng 19. *Mã lỗi và hành vi*

Mã (code)	Khi nào	Máy chủ làm gì	Client nên làm
invalid_sample_rate	Tần số khác 16 kHz	Đóng, mã 1008	Dùng đúng 16000
capacity_exceeded	Đã đủ số luồng tối đa	Đóng, mã 1013	Chờ vài giây rồi thử lại
overloaded	Hơn 30 giây âm thanh chưa xử lý	Đóng, mã 1013	Gửi end thường xuyên hơn; kết nối lại
invalid_frame	Frame có số byte lẻ	Giữ kết nối, bỏ frame	Sửa client
invalid_command	Tin văn bản không hợp lệ	Giữ kết nối	Chỉ gửi tin end
invalid_audio	Engine từ chối âm thanh (ví dụ câu quá dài)	Giữ kết nối	Gửi end thường xuyên hơn
recognition_failed	Không ra văn bản cho câu (hết thời gian, bị hủy)	Giữ kết nối, bỏ câu đó	Tiếp tục câu sau
engine_failure	Lỗi native nặng, hoặc 50 lần từ chối liên tiếp	Đóng, mã 1011	Kết nối lại; báo vận hành

Ngoài ra: kết nối không gửi gì trong 60 giây bị đóng với mã 1001; frame nhị phân lớn hơn 64 KiB bị đóng với mã 1009; client nhận kết quả quá chậm (quá 10 giây) bị ngắt.

4.3 Giao tiếp REST

Bảng 20. *Các thao tác REST*

Thao tác	Đầu vào	Kết quả
POST /jobs	Tệp âm thanh (multipart); tham số model (qwen3-asr hoặc whisper-en); tham số refine (true hoặc false)	Mã 202 kèm jobId và trạng thái ban đầu
GET /jobs/{jobId}	Mã jobId	Trạng thái (QUEUED, RUNNING, DONE, FAILED, CANCELLED), giai đoạn, tiến độ từ 0 đến 1, các đoạn đã xong, văn bản cuối
DELETE /jobs/{jobId}	Mã jobId	Hủy job đang chạy (bật cờ abort trong C++) và xóa tệp tạm

Thao tác	Đầu vào	Kết quả
POST / transcriptions	Một tệp WAV 16 kHz, 16-bit, mono, tối đa 60 giây	JSON gồm văn bản, tần số và thời lượng (ms)
GET /status	Không	Engine, mô hình mặc định, danh sách mô hình, trạng thái refiner, sức chứa (workers, maxSessions, activeSessions, queuedTasks), ready

Bảng 21. *Mã lỗi HTTP (định dạng application/problem+json, RFC 9457)*

Mã	Ý nghĩa
400	Tệp sai định dạng, sai tần số lấy mẫu, hoặc quá dài cho /transcriptions
401	Thiếu hoặc sai khóa API (khi đã bật xác thực)
413	Tệp quá lớn (giới hạn 500 MB)
415	Yêu cầu không phải multipart
503	Mô hình lỗi, hàng đợi job đầy, hoặc mọi slot đang bận (kèm tiêu đề Retry-After khi slot bận)

4.4 Xác thực bằng khóa API

Xác thực mặc định tắt: ai nối được vào máy chủ đều dùng được, phù hợp board trong mạng nội bộ. Khi đặt danh sách khóa (biến STT_API_KEYS, cách nhau bằng dấu phẩy), mọi đường dẫn /api và /ws đòi khóa.

Bảng 22. *Cách gửi khóa API*

Kênh	Cách gửi khóa
REST	Tiêu đề Authorization dạng Bearer, hoặc tiêu đề X-Api-Key
WebSocket từ ứng dụng Android	Một trong hai tiêu đề trên (khuyến nghị)
WebSocket từ trình duyệt	Tham số apiKey trên địa chỉ, vì trình duyệt không đặt được tiêu đề; khóa trong địa chỉ có thể bị proxy ghi lại
Thiếu hoặc sai khóa	REST trả 401 kèm WWW-Authenticate; WebSocket bị từ chối ngay ở bước bắt tay; mỗi lần từ chối tăng metric stt.auth.rejected
Luôn mở, không cần khóa	Trang demo và probe health. Metrics mở mặc định; đặt STT_PROTECT_ACTUATOR để bảo vệ

4.5 Hướng dẫn cho ứng dụng Android

- Dùng AudioRecord với PCM 16-bit, kênh mono, 16.000 Hz: đúng định dạng máy chủ cần, chỉ việc đẩy buffer vào frame nhị phân.
- Gửi mỗi 100 ms một frame và chỉ gửi khi có tiếng nói. Trang demo gửi kèm 0,3 giây đệm phía trước để không cắt đầu từ.
- Tự phát hiện ngừng nói phía client rồi gửi end. Trang demo gửi end sau khoảng 0,8 giây im lặng, và sau 25 giây nếu nói liền.

- Đếm kết quả theo từng lệnh end thật sự có âm thanh; mỗi lệnh nhận đúng một final hoặc một recognition_failed.
- Khi nhận capacity_exceeded hoặc overloaded: kết nối lại có chờ tăng dần, không thử lại dồn dập.
- Đặt hạn chờ kết quả đủ lớn. Với engine jni, một lần end có thể mất nhiều giây trên board (**Chưa test**: chưa có số đo thời gian này).
- Kết nối im lặng quá 60 giây bị đóng; client cần kết nối lại khi cần.
- Spring Boot không chạy trong APK: ứng dụng Android luôn là client, máy chủ đặt trên board, PC hoặc server.

4.6 Hướng dẫn cho hệ thống backend, web khác

- **Tệp hợp dài:** gửi tệp tới /jobs, lưu jobId, hỏi /jobs/{jobId} định kỳ cho đến khi trạng thái DONE rồi lấy văn bản. Kết quả giữ 24 giờ rồi bị xóa, nên cần lưu lại nếu muốn dùng lâu.
- **Proxy ngược (nginx, ...):** bật nâng cấp WebSocket (tiêu đề Upgrade và Connection); đặt proxy_read_timeout lớn hơn 60 giây; trỏ readiness probe vào /actuator/health/readiness; đặt HTTPS ở proxy.
- **Giám sát:** Prometheus lấy /actuator/prometheus. Chỉ số cần theo dõi nhất là stt.queue.wait: tăng nghĩa là máy đang quá tải.
- **Nguồn gốc cho phép (Origin):** mặc định cho mọi nguồn; trước khi mở ra ngoài, đặt danh sách cụ thể bằng stt.websocket.allowed-origins.
- **Load balancer:** node đầy slot trả 503 ở readiness (trạng thái BUSY) để người mới được chuyển sang node khác; liveness luôn UP khi tiến trình còn sống.

4.7 Phiên bản và tương thích

- Phiên bản hiện hành là /api/v1. Đường dẫn cũ /api/stt vẫn dùng được như bí danh nhưng đã deprecated: phản hồi kèm tiêu đề Deprecation và Link trỏ đường dẫn v1 tương ứng. Chưa có ngày gỡ bỏ.
- /api/health đã bỏ; dùng /actuator/health.
- Đường dẫn WebSocket /ws/stt được giữ nguyên.
- Giao thức WebSocket và REST là hợp đồng với client Android: khi đổi phải cập nhật đồng thời README, tệp SKILL.md và bài kiểm thử đầu-cuối.

4.8 Danh sách kiểm tra khi tích hợp

Bảng 23. *Checklist cho nhóm tích hợp*

Số	Việc cần kiểm tra
1	Âm thanh đúng 16 kHz, 16-bit, mono, little-endian; số byte mỗi frame chẵn
2	Gửi lệnh end cho mỗi câu và đếm đúng một kết quả cho mỗi lệnh
3	Xử lý capacity_exceeded và overloaded bằng kết nối lại có chờ
4	Gửi khóa API bằng tiêu đề (không đặt trong địa chỉ nếu tránh được)
5	Hạn chờ phía client lớn hơn thời gian xử lý một lần end
6	Ghi lại lagMs để theo dõi độ trễ thực tế trong môi trường của bạn
7	Dùng /api/v1; không phụ thuộc đường dẫn /api/stt cũ

5. CÀI ĐẶT VÀ VẬN HÀNH

5.1 Lấy mã nguồn

Cài Git, rồi sao chép (clone) kho mã nguồn dưới đây về máy. Nhánh main là nhánh ổn định mới nhất. Thư mục gốc chứa Gradle wrapper (gradlew và gradlew.bat), nên không cần cài Gradle riêng.

https://github.com/DuongMinhHoangHE191087/BocBang_Backend

5.2 Yêu cầu hệ thống

Có ba cấp độ chạy, từ nhẹ đến đầy đủ. Cấp A đủ để người xem báo cáo tự chạy và xem demo mà không cần mô hình.

Bảng 24. *Yêu cầu hệ thống theo cấp độ*

Hạng mục	A. Chạy thử (engine giả)	B. Đầy đủ trên PC	C. Board G520
Mục đích	Xem demo, thử tải, phát triển client	Chạy mô hình thật trên PC	Môi trường thật
Hệ điều hành	Windows, Linux, macOS	Linux x86_64 (để biên dịch .so cho máy host)	Android userdebug có adb root; chroot Ubuntu ARM64 do script dựng
Java	JDK 17	JDK 17	JRE 17 aarch64 (script tải Temurin)
Gradle	Wrapper 9.8.2 tự tải (cần mạng lần đầu)	Như A	Build trên PC
Biên dịch C++	Không cần	CMake từ 3.22, Ninja, trình biên dịch C++17, mã nguồn 3 thư viện C++	Android NDK (android-arm64) hoặc aarch64-linux-gnu-gcc (linux-arm64)
Mô hình	Không cần	4 tệp mô hình trong thư mục STT_MODEL_DIR	Đẩy bằng adb qua biến PUSH_MODELS
ffmpeg	Không cần	Cần khi tệp dài không phải WAV chuẩn	Bản tĩnh ARM64 do script tải
RAM	Heap Java 512 MB; đo thực tế 181 đến 215 MB	Thêm khoảng 1,3 GB (ASR) hoặc 2,5 GB (refiner)	JVM 512 MB cộng mô hình; tổng RAM board: **Cần bổ sung**
Ổ đĩa	JAR 24,5 MiB cộng thư mục job tạm	Thêm dung lượng 4 mô hình: **Cần bổ sung**	Như B; job tạm tối đa 500 MB mỗi tệp
Mạng	Cần lần đầu để tải phụ thuộc	Như A	PC cần mạng để tải rootfs, JRE, ffmpeg; board không cần Internet
CPU	Bất kỳ	x86_64	ARMv8.2-A, 2 nhân A78 và 6 nhân A55 (theo ghi chú cấu hình)

5.3 Chạy thử nhanh với engine giả (cấp A)

1. Cài JDK 17 và Git.
2. Sao chép kho mã và mở thư mục gốc.
3. Đặt biến môi trường STT_ENGINE bằng giá trị mock để dùng engine giả.
4. Chạy tác vụ bootRun bằng Gradle wrapper (Windows dùng gradlew.bat, Linux và macOS dùng gradlew). Lần đầu cần mạng để tải phụ thuộc.
5. Mở trình duyệt tại địa chỉ http://localhost:8099/ . Chọn “File âm thanh” để phát một tệp, hoặc dùng micro (trình duyệt chỉ cho dùng micro trên localhost hoặc https).
6. Để chạy kiểm thử tự động, chạy tác vụ test; không cần mô hình hay thư viện .so. Trên Windows nên thêm tùy chọn không dùng daemon vì daemon Gradle từng biến mất giữa chừng.

> **Lưu ý về cổng**
> Cổng mặc định cấu hình thực tế là 8099 (application.yml và script triển khai). Các ví dụ trong README còn ghi 8080; khi làm theo README hãy đổi sang 8099 hoặc đặt SERVER_PORT bằng 8080.

Kết quả mong đợi: máy chủ khởi động trong khoảng 4 giây và trang /api/v1/stt/status báo ready (**Đã đo** trên máy phát triển với engine giả).

5.4 Chạy đầy đủ với mô hình thật trên PC (cấp B)

1. Chuẩn bị mã nguồn ba thư viện C++ (transcribe.cpp, whisper.cpp, llama.cpp) và chỉ đường dẫn bằng các biến TRANSCRIBE_CPP_DIR, WHISPER_CPP_DIR, LLAMA_CPP_DIR. Đường dẫn mặc định trong CMake trỏ về máy của lập trình viên cũ nên bắt buộc phải đặt các biến này.
2. Biên dịch ba thư viện JNI cho máy host bằng script scripts/build-native.sh với đối số host. Kết quả nằm ở native/dist/host.
3. Đặt STT_MODEL_DIR trỏ tới thư mục chứa bốn tệp mô hình (xem bảng ở mục 3.7).
4. Chạy bootRun như cấp A, nhưng không đặt STT_ENGINE bằng mock.
5. Kiểm tra /api/v1/stt/status và thử bằng web demo hoặc công cụ tools/load-test.mjs (cần Node 22 trở lên).

Chưa kiểm chứng

Các bước cấp B chưa được chạy trong phiên lập báo cáo vì môi trường thiếu thư viện .so và mô hình: **Chưa test**. Hai bài kiểm thử JNI sẽ tự chạy khi đã có .so host và mô hình.

5.5 Triển khai lên board MediaTek G520 (cấp C)

1. Cắm cáp ADB; board ở chế độ userdebug để dùng được adb root. Lệnh chroot cần quyền root.
2. Chạy script scripts/deploy-board.sh trên PC (đối số mặc định linux-arm64). Lần đầu đặt biến PUSH_MODELS trỏ tới thư mục chứa mô hình để đẩy cả mô hình lên board.
3. Script tự làm: biên dịch ba thư viện .so nếu chưa có; đóng gói JAR; tải Ubuntu Base ARM64, JRE 17 và ffmpeg về PC; dựng chroot trên board; đẩy JAR, .so và mô hình; sinh tệp run.sh.
4. Nếu cần truy cập từ PC, chuyển tiếp cổng 8099 bằng adb forward.
5. Chạy run.sh trên board. JVM chạy với heap từ 64 MB đến 512 MB, 4 thread tính toán cho mô hình, và bật unload-before-refine.
6. Kiểm tra /api/v1/stt/status và /actuator/health/readiness từ PC.

Bảng 25. *Bố trí thư mục trên board*

Đường dẫn	Nội dung
/data/local/tmp/bocbang	Thư mục gốc của hệ thống trên board; chứa run.sh
.../models	Các tệp mô hình (gắn vào chroot)
.../jobs	Thư mục tạm của job tệp dài (gắn vào chroot)
.../rootfs	Hệ thống tệp Ubuntu ARM64 cùng JRE 17 và ffmpeg

Chưa kiểm chứng

Quy trình triển khai board có script đầy đủ nhưng chưa được chạy trong phiên lập báo cáo (không có board): **Chưa test**. Tệp .so và script thực thi phải đặt trong /data/local/tmp hoặc thư mục home của Termux, không đặt trên /sdcard vì phân vùng đó gắn noexec.

5.6 Cấu hình quan trọng

Cấu hình đọc từ application.yml hoặc biến môi trường. Với các khóa có biến riêng trong tệp cấu hình, dùng tên biến ghi ở cột giữa; các khóa còn lại đặt trong application.yml hoặc theo quy tắc đặt tên biến môi trường của Spring Boot.

Bảng 26. *Cấu hình thường dùng*

Khóa	Biến môi trường	Mặc định	Ý nghĩa
server.port	SERVER_PORT	8099	Cổng HTTP và WebSocket
stt.engine	STT_ENGINE	jni	jni (mô hình thật) hoặc mock (engine giả)
stt.pool.workers	STT_POOL_WORKERS	0 (CPU trừ 1)	Số thread nhận dạng
stt.pool.max-sessions		0 (workers nhân hệ số engine)	Số luồng tối đa; jni hệ số 1, mock hệ số 4
stt.session.max-buffered-audio	STT_MAX_BUFFERED_AUDIO	30s	Âm thanh chờ tối đa của một client
stt.session.max-batch-audio		500ms	Âm thanh gộp tối đa mỗi lần gọi mô hình
stt.threads	STT_THREADS	4	Thread tính toán trong mỗi lần chạy mô hình
stt.default-model	STT_DEFAULT_MODEL	qwen3-asr	Mô hình nhận dạng mặc định
stt.unload-before-refine	STT_UNLOAD_BEFORE_REFINE	true	Giải phóng ASR trước khi nạp refiner
stt.refiner.enabled	STT_REFINER_ENABLED	true	Bật sửa dấu câu bằng mô hình ngôn ngữ
stt.security.api-keys	STT_API_KEYS	Rỗng (tắt)	Danh sách khóa API
stt.security.protect-actuator	STT_PROTECT_ACTUATOR	false	Đòi khóa cho metrics (probe health vẫn mở)
stt.websocket.allowed-origins		*	Nguồn gốc được phép
stt.max-utterance-seconds		60	Độ dài tối đa một câu
stt.jobs.max-queued		20	Số job chờ tối đa
stt.jobs.result-ttl		24h	Thời gian giữ kết quả job
Thư mục mô hình	STT_MODEL_DIR	/data/local/tmp/bocbang/models	Nơi đặt các tệp mô hình
Thư mục thư viện .so	STT_NATIVE_LIB_DIR	Tự dò native/dist	Nơi đặt ba thư viện JNI
Thư mục job	STT_JOBS_DIR	Thư mục tạm của hệ thống	Nơi ghi tệp job
Lệnh ffmpeg	STT_FFMPEG_BIN	ffmpeg	Đường dẫn chương trình ffmpeg

5.7 Vận hành hằng ngày

Bảng 27. *Chín metric riêng của hệ thống*

Metric	Ý nghĩa
stt.sessions.active	Số luồng đang hoạt động
stt.sessions.max	Số luồng tối đa cho phép
stt.sessions.rejected	Số kết nối bị từ chối vì đầy
stt.inference.workers	Số thread nhận dạng
stt.inference.active	Số tác vụ đang chạy mô hình
stt.inference.queued	Số tác vụ đang chờ
stt.inference.duration	Thời gian mỗi lần chạy mô hình
stt.queue.wait	Thời gian chờ trong hàng đợi; chỉ số quan trọng nhất, tăng nghĩa là quá tải
stt.auth.rejected	Số lần từ chối do khóa API

- **Liveness** (/actuator/health/liveness): luôn UP khi tiến trình còn sống, không phụ thuộc tải.
- **Readiness** (/actuator/health/readiness): trả 503 với trạng thái BUSY khi đầy slot; đây là bận, không phải hỏng.
- **Tắt máy chủ:** chờ yêu cầu đang chạy tối đa 20 giây; thư mục job tạm được dọn khi tắt và quét lại khi khởi động.
- **Nhật ký:** thông báo lỗi chi tiết (đường dẫn mô hình, đầu ra ffmpeg, lỗi native) chỉ có trong log máy chủ; client chỉ nhận mô tả chung.

5.8 Xử lý sự cố thường gặp

Bảng 28. *Triệu chứng, nguyên nhân và cách xử lý*

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Máy chủ không khởi động, thiếu cấu hình stt.models	Engine jni cần khai báo mô hình	Dùng STT_ENGINE=mock khi chưa có mô hình, hoặc khai báo đủ mô hình
Nhận dạng lỗi recognition_failed ngay từ câu đầu	Thiếu tệp .so hoặc tệp mô hình; lỗi chỉ lộ lúc nhận dạng, không lộ lúc khởi động	Kiểm tra STT_MODEL_DIR, STT_NATIVE_LIB_DIR và log máy chủ
capacity_exceeded hoặc 503 kèm Retry-After	Đã đủ số luồng tối đa	Thử lại sau; chỉ tăng max-sessions sau khi đo sức chứa
overloaded giữa chừng	Client gửi nhanh hơn máy chủ xử lý	Gửi end thường xuyên hơn; xem stt.queue.wait
Job báo FAILED với thông báo chung	Lỗi chi tiết chỉ nằm trong log	Đọc log máy chủ; thường do thiếu ffmpeg hoặc tệp hỏng
413 khi tải tệp	Tệp lớn hơn 500 MB	Cắt tệp hoặc nén âm thanh

6. HIỆU NĂNG, SỨC CHỨA VÀ ĐÁNH GIÁ

6.1 Nguyên tắc và phạm vi đo

Báo cáo chỉ ghi "Đã đo" cho số liệu đo thật trong phiên lập báo cáo. Những gì chưa đo được ghi "Chưa test" và không được thay bằng ước lượng. Quan trọng nhất: môi trường lập báo cáo là máy phát triển Windows, chưa có thư viện .so và mô hình, nên mọi phép đo tải đều dùng engine giả.

> **Phạm vi của số đo**
> Các số đo ở mục 6.4 cho thấy lớp giao tiếp và điều phối (giới hạn luồng, độ trễ xếp hàng, bộ nhớ khung ứng dụng). Chúng không phản ánh tốc độ của mô hình thật và không dùng để suy ra sức chứa của board G520. README của dự án cũng cảnh báo: phải đo sức chứa bằng tiếng nói thật, vì im lặng giải mã nhanh hơn nhiều lần.

6.2 Môi trường đo

Bảng 29. *Cấu hình máy và phần mềm khi đo (29/10/2026)*

Hạng mục	Giá trị
Máy	Intel Core i5-8300H 2,30 GHz, 4 nhân 8 luồng; RAM 13,9 GB
Hệ điều hành	Windows 11 Enterprise (phiên bản 10.0.26100)
Java	Eclipse Temurin 17.0.20; JVM chạy với heap 64 MB đến 512 MB (cùng tham số dùng trên board)
Phần mềm	Spring Boot 4.1.2; Gradle 9.8.2; Node 24.18 cho công cụ thử tải
Engine	mock (engine giả); số luồng tối đa mặc định là 28 (7 worker nhân 4)
Âm thanh gửi vào	Im lặng 16 kHz (mặc định của công cụ khi không có tệp tiếng nói)
Phiên bản mã	Nhánh main, commit 711fd68
Board G520	Không có trong phiên đo

6.3 Kết quả kiểm thử tự động

Toàn bộ bộ kiểm thử được chạy bằng Gradle trên máy phát triển. Kết quả: 117 test, 115 đạt, 2 bỏ qua, 0 lỗi. Tổng thời gian chạy các test là 15,6 giây; cả quá trình biên dịch, kiểm thử và đóng gói JAR khởi động nguội mất 1 phút 14 giây. CI trên GitHub chạy lại toàn bộ bộ test cho mỗi pull request.

Bảng 30. *Kết quả kiểm thử theo nhóm*

Nhóm	Tệp kiểm thử	Số test	Kết quả
Điều phối đa luồng	InferenceExecutor, RuntimeProperties, SessionLimiter, SessionPipeline	31	Đạt hết
WebSocket handler	SpeechWebSocketHandlerTest	10	Đạt hết
Bảo mật	ApiKeyFilter, SttSecurityEndToEnd, SttActuatorProtectionEndToEnd	22	Đạt hết

Nhóm	Tệp kiểm thử	Số test	Kết quả
Dịch vụ nhận dạng và job	TranscriptionService, TranscriptionJobService, TranscriptionJobHardening	20	Đạt hết
Engine (hợp đồng, giả, JNI)	FakeSpeechToTextEngineContract, MockSpeechEngine, JniSpeechEngine	19	17 đạt, 2 bỏ qua
Xử lý lỗi API	ApiExceptionHandlerTest	2	Đạt hết
Đầu-cuối (server thật, cổng ngẫu nhiên)	SttEndToEndTest	13	Đạt hết
Tổng		117	115 đạt, 2 bỏ qua, 0 lỗi

Hai test bị bỏ qua

Hai test của engine JNI (chạy Silero VAD và Whisper thật; nạp ba thư viện .so vào một JVM mà không xung đột ký hiệu ggml) tự bỏ qua khi chưa có .so host và mô hình. Đây là phần kiểm chứng quan trọng nhất của lớp JNI nhưng chưa được chạy: **Chưa test**.

6.4 Đo đồng thời với engine giả

Công cụ tools/load-test.mjs mở nhiều kết nối WebSocket, mỗi kết nối phát âm thanh ở tốc độ thật trong 15 giây và ghi lại độ trễ lagMs. Ba kịch bản được chạy lần lượt trên cùng một máy chủ vừa khởi động.

Bảng 31. *Kết quả ba kịch bản đo đồng thời (engine giả)*

Kịch bản	Kết nối vào	Được nhận	Bị từ chối	Kết quả nhận	lagMs p50 / p95 / p99 / tối đa	Thời gian (giây)
Dưới mức tối đa	8	8	0	488	0 / 0 / 0 / 0	16,8
Đúng mức tối đa	28	28	0	1.708	0 / 0 / 0 / 0	16,8
Vượt mức tối đa	40	28	12	1.708	0 / 0 / 0 / 1	16,9

Bảng 32. *Chỉ số khác đo được trong cùng đợt*

Chỉ số	Giá trị	Mức xác thực
Thời gian khởi động Spring Boot	4,0 giây	**Đã đo**
Thời gian đến khi readiness trả 200	5,5 giây (gồm khởi động tiến trình Java)	**Đã đo**
Bộ nhớ tiến trình Java khi rảnh	215 MB	**Đã đo**
Bộ nhớ tiến trình Java sau mỗi kịch bản tải	181 đến 191 MB	**Đã đo**
Sức chứa báo ở /status (engine giả)	7 worker, tối đa 28 luồng	**Đã đo**

Ý nghĩa của kết quả

- **Giới hạn luồng chính xác:** với 40 kết nối vào máy chủ giới hạn 28, đúng 12 kết nối bị từ chối ngay bằng capacity_exceeded; 28 kết nối còn lại không bị ảnh hưởng.
- **Giữ nhịp thời gian thực:** 15 giây âm thanh xử lý xong trong khoảng 16,8 đến 16,9 giây (gồm mở và đóng kết nối), không bị dồn.
- **Lớp điều phối không là nút cổ chai:** độ trễ xếp hàng gần như bằng 0 ms ở cả 28 kết nối.
- **Bộ nhớ:** tiến trình Java dùng khoảng 181 đến 215 MB, thấp hơn nhiều so với giới hạn heap 512 MB dùng trên board.

> **Không suy ra sức chứa board từ bảng này**
> Engine giả gần như không tốn CPU và âm thanh gửi vào là im lặng. Con số 28 luồng ở trên là mức mặc định của engine giả trên máy 8 luồng, không liên quan đến năng lực của board G520 với mô hình thật.

6.5 Độ chính xác nhận dạng (WER)

Chưa có bất kỳ số đo độ chính xác nào trong kho hay trong phiên lập báo cáo. Bảng dưới liệt kê các nội dung cần đo.

Bảng 33. *Các chỉ tiêu độ chính xác*

Nội dung đo	Kết quả	Mức xác thực
Tỷ lệ lỗi từ (WER) tiếng Việt với Qwen3-ASR	Chưa đo	**Chưa test**
Tỷ lệ lỗi từ (WER) tiếng Anh với Whisper small EN	Chưa đo	**Chưa test**
Chất lượng văn bản sau refiner (dấu câu, viết hoa, lỗi từ)	Chưa đo	**Chưa test**
Ảnh hưởng của VAD đến việc cắt mất chữ ở đầu hoặc cuối đoạn	Chưa đo	**Chưa test**
Ngưỡng WER chấp nhận được	Chưa có	**Cần bổ sung**

Phương pháp đo đề xuất

1. Chọn khoảng 30 đến 50 tệp tiếng nói thật (họp thực tế, nhiều giọng, có tạp âm), kèm bản gõ chuẩn do người nghe lại.
2. Chạy từng tệp qua hệ thống hai lần: tắt refiner (refine bằng false) và bật refiner (refine bằng true).
3. Tính WER bằng tổng số từ thay thế, thiếu và thừa chia cho tổng số từ chuẩn, cho từng nhóm tệp.
4. So sánh trước và sau refiner; ghi lại các lỗi điển hình (tên riêng, số, từ chuyên ngành).

5. Nghiệp vụ đặt ngưỡng chấp nhận và quyết định có cần tinh chỉnh mô hình hay không.

6.6 Sức chứa kết nối trên board G520

Đây là câu hỏi lãnh đạo quan tâm nhất: phần cứng này phục vụ tối đa bao nhiêu kết nối, và xử lý ra sao. Câu trả lời gồm hai phần: giới hạn cấu hình (đọc được từ mã) và sức chịu tải thật (chưa đo).

Giới hạn theo cấu hình mặc định

Bảng 34. *Các đại lượng tính từ cấu hình, board 8 nhân, engine jni*

Đại lượng	Công thức, giá trị	Mức xác thực
Số thread nhận dạng	Số CPU trừ 1 = 7	**Theo mã nguồn**
Số luồng thời gian thực tối đa	7 worker nhân 1 luồng mỗi worker = 7	**Theo mã nguồn**
Số phép tính mô hình chạy cùng một lúc	1 (CpuExecutionGate)	**Theo mã nguồn**
Số job tệp dài chạy cùng một lúc	1, cộng tối đa 20 job chờ	**Theo mã nguồn**
Số kết nối mở tối đa theo Tomcat	8.192, nhưng bị chặn ở 7 luồng nhận dạng	**Theo mã nguồn**
Thread tính toán trong mỗi lần chạy mô hình	4	**Theo mã nguồn**

Kết luận theo cấu hình: tối đa 7 kết nối nhận dạng thời gian thực cùng lúc (khi JVM thấy 8 CPU); kết nối thứ 8 nhận capacity_exceeded. Đây là mức do nhóm phát triển đặt thận trọng, README ghi rõ là chưa được đo, nên chưa thể coi là sức chứa đã chứng minh. Số thực tế có thể thấp hơn hoặc cao hơn.

Cách hệ thống xử lý khi nhiều người nói cùng lúc

Engine jni gom âm thanh của từng người. Khi một người nói xong và gửi lệnh end, câu của họ xếp vào hàng đợi của cổng native, rồi các câu chạy lần lượt. Hệ quả: độ trễ của người cuối hàng bằng tổng thời gian xử lý các câu đứng trước. Job tệp dài và refiner cũng đi qua cùng cổng này, nên một job đang chạy có thể làm các câu thời gian thực chờ lâu hơn (**Chưa test**).

Công thức ước lượng để lập kế hoạch

Gọi R là hệ số thời gian thực của mô hình trên board (giây xử lý trên mỗi giây âm thanh) và p là tỷ lệ thời gian một người thực sự nói. Mỗi người chiếm trung bình p nhân R phần thời gian của cổng native. Để hàng đợi không dồn, tổng tải phải dưới 1; để độ trễ ổn định nên chừa dư, ví dụ dùng tối đa 70%. Số người nói đồng thời hợp lý xấp xỉ 0,7 chia cho tích p và R, và không vượt giới hạn cấu hình 7.

Bảng 35. *Bảng minh họa công thức (GIẢ ĐỊNH, KHÔNG PHẢI SỐ ĐO)*

Hệ số thời gian thực R (giả định)	p = 0,3 (nói 30% thời gian)	p = 0,5 (nói 50% thời gian)
R = 0,2 (nhanh gấp 5 lần thời gian thực)	Tính ra 11, giới hạn cấu hình còn 7	Tính ra 7
R = 0,5 (nhanh gấp 2 lần thời gian thực)	4	2
R = 1,0 (đúng bằng thời gian thực)	2	1

Lưu ý quan trọng về bảng trên

Các giá trị R trong bảng là giả định để minh họa cách tính, không phải số đo. Hệ số R thật của Qwen3-ASR trên G520: **Chưa test**. Không dùng bảng này để cam kết số người dùng; chỉ dùng sau khi thay R bằng số đo thật.

Quy trình đo sức chứa thật (đề xuất)

1. Triển khai bản mới nhất lên board; tắt các tiến trình nền nặng, vì chúng làm số đo sai nhiều lần.
2. Chuẩn bị tệp WAV tiếng nói thật 16 kHz, mono, 16-bit, dài từ 60 giây trở lên.
3. Từ PC, chạy công cụ thử tải tools/load-test.mjs với số kết nối tăng dần 1, 2, 3, 4, 5, 6, 7; mỗi lần 40 giây; luôn bật tùy chọn gửi end định kỳ (ví dụ mỗi 8 giây), bắt buộc với engine jni.
4. Ghi p50, p95, p99 của lagMs, số kết nối bị ngắt hoặc từ chối, giá trị stt.queue.wait, RAM và nhiệt độ board vào biểu mẫu ở Phụ lục C.
5. Chọn số kết nối lớn nhất mà p95 của lagMs còn dưới ngưỡng chấp nhận (README gợi ý ví dụ 2 giây), rồi đặt stt.pool.max-sessions thấp hơn mức đó một chút.

6.7 Bộ nhớ

Bảng 36. *Nhu cầu bộ nhớ chính*

Thành phần	Dung lượng	Mức xác thực
Heap JVM trên board	Tối đa 512 MB (tham số khởi chạy của script)	**Theo mã nguồn**
Mô hình ASR Qwen3-ASR 1.7B Q4_K_M	Khoảng 1,3 GB	**Theo mã nguồn**
Refiner Qwen3-4B Instruct Q4_K_M	Khoảng 2,5 GB	**Theo mã nguồn**
Đỉnh khi refine (ASR được giải phóng trước)	Khoảng 2,5 GB thay vì 3,8 GB	**Theo mã nguồn**

Thành phần	Dung lượng	Mức xác thực
Âm thanh một câu thời gian thực ở mức tối đa 60 giây	Khoảng 1,9 MB PCM và 3,8 MB khi đổi sang số thực; khoảng 40 MB cho 7 phiên	Tính toán từ cấu hình
Âm thanh chờ tối đa của mỗi client	Khoảng 1 MB (30 giây)	Tính toán từ cấu hình
Cửa sổ VAD của job tệp dài	Khoảng 19 MB heap mỗi cửa sổ 5 phút	**Theo mã nguồn**
Tổng RAM của board	Chưa có thông tin	**Cần bổ sung**

Cơ chế unload-before-refine là đánh đổi có chủ ý: giải phóng ASR trước khi nạp refiner để không vượt RAM, nhưng câu thời gian thực kế tiếp phải chờ nạp lại mô hình, có thể mất nhiều giây (**Chưa test**: cần số đo để chọn giữ hay tắt cơ chế này).

6.8 Giới hạn cứng và hành vi khi quá tải

Bảng 37. *Các giới hạn mặc định của hệ thống*

Giới hạn	Giá trị	Khi vượt
Frame nhị phân WebSocket	64 KiB	Đóng, mã 1009
Tin văn bản WebSocket	4 KiB	Bị từ chối
Độ dài một câu thời gian thực	60 giây	invalid_audio, giữ kết nối
Âm thanh chờ mỗi client	30 giây	overloaded, đóng 1013
Kết nối không gửi gì	60 giây	Đóng, mã 1001
Client nhận chậm	10 giây hoặc 512 KiB	Ngắt client
Tệp REST đồng bộ	WAV tối đa 60 giây	Lỗi 400
Tệp tải lên job	500 MB	Lỗi 413
Hàng đợi job	1 chạy, 20 chờ	Lỗi 503 trước khi nhận tệp
Thời gian giữ kết quả job	24 giờ	Xóa
Thời gian tối đa mỗi lần chạy ASR, refiner, ffmpeg	5 phút, 10 phút, 15 phút	Hủy; job FAILED
Engine từ chối liên tiếp	50 lần	engine_failure, đóng phiên
Tomcat	200 thread, 8.192 kết nối, hàng chờ 100	Theo Tomcat

6.9 Đánh giá tổng hợp

Bảng 38. *Tổng hợp đánh giá theo khía cạnh*

Khía cạnh	Đánh giá	Căn cứ	Mức xác thực
Đúng chức năng (điều phối, giao thức, bảo mật)	Tốt	115 trên 115 test chạy được đều đạt, không lỗi	**Đã kiểm thử**
Chịu tải đồng thời (lớp điều phối)	Tốt trong điều kiện thử	40 kết nối: nhận 28, từ chối 12, trễ gần 0 ms	**Đã đo**
Độ chính xác nhận dạng	Chưa kết luận	Chưa có số đo WER	**Chưa test**
Hiệu năng mô hình trên board	Chưa kết luận	Chưa đo tốc độ và độ trễ với mô hình thật	**Chưa test**
Triển khai lên board	Chưa kết luận	Có script đầy đủ; chưa chạy trong phiên này	**Chưa test**
Khả năng vận hành, giám sát	Tốt	Probe, 9 metric, tắt êm, tự dọn tệp tạm	**Theo mã nguồn**
Tài liệu	Đầy đủ	README 271 dòng, 14 quyết định kiến trúc, bảng mã lỗi	**Theo mã nguồn**

7. BẢO MẬT VÀ AN TOÀN VẬN HÀNH

7.1 Biện pháp hiện có

Bảng 39. *Các biện pháp bảo mật trong hệ thống*

Biện pháp	Mô tả	Mức xác thực
Khóa API tùy chọn	Bộ lọc duy nhất ApiKeyFilter bảo vệ /api và /ws; không rải kiểm tra vào controller	**Đã kiểm thử**
Chống vượt bộ lọc bằng path parameter	Bộ lọc xét đường dẫn servlet, không xét địa chỉ thô, nên các dạng như /api;x=1 không qua mặt được	**Đã kiểm thử**
Không lộ lỗi nội bộ	Đường dẫn mô hình, đầu ra ffmpeg, lỗi native chỉ vào log; client nhận mô tả chung	**Đã kiểm thử**
Từ chối sớm	Hàng đợi job đầy bị từ chối trước khi nhận tệp 500 MB	**Đã kiểm thử**
Kiểm tra định dạng âm thanh	Âm thanh sau chuyển đổi phải đúng 16-bit mono; nếu không job báo lỗi thay vì nhận dạng ra rác	**Đã kiểm thử**
Dọn tệp tạm	Xóa thư mục job khi tắt, quét thư mục sót khi khởi động, xóa kết quả sau 24 giờ	**Theo mã nguồn**
Không gọi mạng ra ngoài	Mã nguồn không có thư viện hay lời gọi HTTP ra ngoài; dữ liệu giọng nói không rời máy chủ	**Theo mã nguồn**

7.2 Khoảng trống và khuyến nghị

Bảng 40. *Khoảng trống bảo mật cần xử lý trước khi mở rộng*

Vấn đề	Hiện trạng	Khuyến nghị
Không mã hóa đường truyền	Khóa API và âm thanh đi dạng rõ	Đặt sau proxy HTTPS; chưa thử khóa API sau proxy HTTPS: **Chưa test**
Nguồn gốc cho phép mặc định là *	Mọi nguồn được kết nối WebSocket	Đặt stt.websocket.allowed-origins theo danh sách cụ thể
Metrics mặc định mở	Ai nối được máy chủ đều xem được	Bật STT_PROTECT_ACTUATOR và cấp khóa cho Prometheus
Khóa API trong địa chỉ (WebSocket trình duyệt)	Có thể bị proxy ghi lại	Dùng tiêu đề với client Android
Không có phân quyền hoặc giới hạn tốc độ theo khóa	Mọi khóa hợp lệ có quyền như nhau	Cân nhắc nếu nhiều nhóm cùng dùng
Chưa kiểm thử xâm nhập độc lập	Chưa có	**Chưa test**

8. CHẤT LƯỢNG MÃ NGUỒN VÀ QUY TRÌNH

8.1 Quy mô mã nguồn

Bảng 41. *Quy mô mã nguồn và tài liệu (đếm từ kho, nhánh main)*

Thành phần	Số tệp	Số dòng	Mức xác thực
Java chạy thật (src/main)	39	3.266	**Đã đo**
Java kiểm thử (src/test)	21	2.735	**Đã đo**
C++ JNI, CMake, danh sách xuất	6	639	**Đã đo**
Web demo (HTML, CSS, JS)	4	1.108	**Đã đo**
Script và công cụ (build, triển khai, thử tải)	3	346	**Đã đo**
Tài liệu (README, quyết định kiến trúc, CLAUDE.md)	3	361	**Đã đo**

Tỷ lệ mã kiểm thử trên mã chạy thật là 0,84 dòng trên mỗi dòng (2.735 chia 3.266). Gói runtime, nơi chứa logic đa luồng khó nhất, có 31 test riêng cùng 13 test đầu-cuối chạy server thật.

8.2 Quy trình phát triển

Bảng 42. *Quy trình tự động hóa và kiểm soát*

Hạng mục	Hiện trạng
CI (GitHub Actions)	Chạy khi mở pull request và khi đẩy lên main; Ubuntu, JDK 17 Temurin; giới hạn 20 phút; giữ báo cáo test 7 ngày khi lỗi. Lần chạy mới nhất trên main thành công.
Cập nhật phụ thuộc	Dependabot hằng tuần cho Gradle và GitHub Actions, tối đa 3 pull request mở; đã gộp các bản nâng Gradle wrapper lên 9.8.2 và ba GitHub Actions.
Lịch sử và cộng tác	38 commit; có pull request và ghi chép quyết định kiến trúc.
Kiểm thử không cần mô hình	CI và máy phát triển chạy được toàn bộ test mà không cần .so hay mô hình.

8.3 Điểm mạnh và điểm cần cải thiện

Điểm mạnh

- Kiến trúc phân lớp rõ ràng; thêm engine mới không phải sửa phần giao tiếp, có bộ test hợp đồng kèm theo.
- Logic đa luồng và xử lý lỗi được kiểm thử dày và ghi lại lý do từng quyết định.
- Có sẵn health probe, metrics, tắt êm, dọn tệp tạm; lỗi không lộ thông tin nội bộ.
- Tài liệu tiếng Việt chi tiết cho người dùng và người phát triển.

Điểm cần cải thiện (đã kiểm tra trong kho)

- **Chưa có số đo với mô hình thật trên board:** sức chứa, độ trễ, độ chính xác đều chưa đo.
- **Đường dẫn cứng trong cấu hình biên dịch:** tệp native/CMakeLists.txt trỏ tới thư mục máy của lập trình viên cũ cho ba thư viện C++, và script build-native.sh mặc định tìm NDK ở đường dẫn cá nhân; người mới bắt buộc phải đặt biến môi trường.
- **Phiên bản thư viện C++ chưa được ghim:** ba thư viện nằm ngoài kho, không dùng submodule hay ghi phiên bản.
- **Mô hình và .so không nằm trong kho:** chưa có hướng dẫn nguồn tải và dung lượng từng tệp.
- **README ghi cổng 8080** trong khi cấu hình thực tế là 8099.

8.4 Việc còn nợ do nhóm phát triển ghi nhận

Bảng 43. *Các việc chưa làm, theo docs/quyet-dinh-kien-truc.md*

Việc còn nợ	Cần gì để làm
Đo sức chứa engine jni trên board G520 rồi đặt stt.pool.max-sessions	Board và tiếng nói thật
Xác nhận max-buffered-audio 30 giây với thời gian một lần end thật	Số đo trên board
Thử khóa API sau proxy HTTPS	Môi trường có proxy
Bật bảo vệ metrics ở môi trường thật và cấp khóa cho Prometheus	Quyết định vận hành
Job có refine gỡ mô hình ASR dùng chung với thời gian thực	Số đo để chọn unload-before-refine
Upload job ghi đĩa hai lần	Đổi chữ ký hàm submit của phần job
Ngày gỡ đường dẫn /api/stt cũ	Biết client Android đã chuyển sang /api/v1 chưa
Readiness chưa phản ánh việc thiếu tệp mô hình	Thêm kiểm tra khi khởi động

9. KẾT LUẬN VÀ ĐỀ XUẤT

9.1 Kết luận

BocBang Backend đã có phần "xương sống" hoàn chỉnh và được kiểm chứng tốt: giao thức WebSocket và REST rõ ràng, lớp điều phối đa luồng bảo vệ máy khỏi quá tải, bảo mật và xử lý lỗi chặt chẽ, vận hành có probe và metrics. Với engine giả, hệ thống khởi động trong 4 giây và từ chối chính xác các kết nối vượt mức.

Tuy nhiên, hai nội dung quyết định giá trị thực của sản phẩm vẫn chưa được chứng minh bằng số đo: độ chính xác nhận dạng và sức chứa của board G520 với mô hình thật. Giới hạn cấu hình mặc định là 7 luồng thời gian thực, nhưng vì mọi phép tính mô hình chạy lần lượt từng cái một, con số này chỉ là mức thận trọng, không phải sức chứa đã kiểm chứng. Đề nghị không cam kết số người dùng hoặc chỉ tiêu chất lượng cho đến khi hoàn tất các phép đo ở mục 9.2.

9.2 Lộ trình đề xuất

Bảng 44. *Các việc ưu tiên*

Ưu tiên	Việc cần làm	Kết quả	Cần chuẩn bị
1	Đo sức chứa trên board bằng tiếng nói thật (mục 6.6)	Số luồng tối đa có căn cứ; đặt max-sessions	Board, tệp tiếng nói thật
2	Đo độ chính xác WER với và không refiner (mục 6.5)	Chỉ tiêu chất lượng	Bộ tệp có bản gõ chuẩn
3	Đo thời gian một lần end và thời gian nạp lại mô hình sau refine	Xác nhận max-buffered-audio và unload-before-refine	Board đã triển khai
4	Chạy hai test JNI đang bị bỏ qua trên máy có .so và mô hình	Xác nhận không xung đột ký hiệu ggml	Thư viện .so và mô hình
5	Thử HTTPS với khóa API; bật bảo vệ metrics	Sẵn sàng mở ngoài mạng nội bộ	Proxy HTTPS
6	Sửa tài liệu cổng, đường dẫn cứng; ghim phiên bản thư viện C++	Người ngoài dựng lại được môi trường	Quyết định của nhóm
7	Quyết định ngày gỡ /api/stt cũ	Dọn đường dẫn lỗi thời	Xác nhận client Android

9.3 Thông tin cần lãnh đạo và nghiệp vụ cung cấp

- Mục tiêu kinh doanh và số người dùng đồng thời cần phục vụ (**Cần bổ sung**).
- Ngưỡng WER chấp nhận được cho tiếng Việt và tiếng Anh (**Cần bổ sung**).
- Ngưỡng độ trễ chấp nhận được, ví dụ p95 của lagMs (**Cần bổ sung**).
- Cấu hình board: tổng RAM, lưu trữ, tản nhiệt (**Cần bổ sung**).
- Bộ tệp tiếng nói chuẩn kèm bản gõ để đo (**Cần bổ sung**).

PHỤ LỤC A. THUẬT NGỮ

Thuật ngữ	Giải thích
STT, ASR	Nhận dạng giọng nói, chuyển lời nói thành văn bản
VAD	Phát hiện đoạn có tiếng nói (Silero VAD), dùng để cắt bỏ phần im lặng
WER	Tỷ lệ lỗi từ: số từ sai (thay thế, thiếu, thừa) chia cho tổng số từ chuẩn; càng thấp càng tốt
RTF (hệ số thời gian thực)	Thời gian xử lý chia cho thời lượng âm thanh; nhỏ hơn 1 nghĩa là nhanh hơn thời gian thực
JNI	Cầu nối cho phép Java gọi mã C++ trong cùng tiến trình
GGML, GGUF	Thư viện tính toán và định dạng tệp mô hình AI chạy hiệu quả trên CPU
Lượng tử hóa (Q4_K_M)	Nén mô hình xuống khoảng 4 bit để giảm RAM và tăng tốc, đổi lại mất chút chất lượng
PCM 16-bit	Âm thanh thô, mỗi mẫu 2 byte; 16 kHz nghĩa là 16.000 mẫu mỗi giây
partial, final	Kết quả tạm và kết quả chốt của một câu
lagMs	Thời gian âm thanh mới nhất chờ trong máy chủ trước khi có kết quả (ms)
p50, p95, p99	Giá trị mà 50%, 95%, 99% phép đo nằm dưới; p95 phản ánh trải nghiệm tệ của đa số người dùng
Slot, luồng	Một suất phục vụ đồng thời; hết slot thì kết nối mới bị từ chối
Refiner	Mô hình ngôn ngữ thêm dấu câu, viết hoa và sửa lỗi cho văn bản nhận dạng
Liveness, readiness	Hai loại probe: còn sống, và sẵn sàng nhận người mới
RFC 9457	Chuẩn định dạng lỗi HTTP (application/problem+json)
chroot	Môi trường Linux cô lập chạy trên Android, dùng để chạy Java và ffmpeg trên board
ADB	Công cụ kết nối PC với thiết bị Android qua cáp USB

PHỤ LỤC B. BẢN ĐỒ KHO MÃ NGUỒN

Đường dẫn trong kho	Nội dung
README.md	Hướng dẫn dùng, API, cấu hình, triển khai
docs/quyet-dinh-kien-truc.md	14 quyết định kiến trúc và việc còn nợ
CLAUDE.md	Quy ước cho người viết mã và cách kiểm tra
.claude/skills/stt-backend/SKILL.md	Tóm tắt kiến trúc và giao thức cho trợ lý lập trình
src/main/java/com/bocbang/backend	Mã nguồn Java: api, websocket, runtime, security, stt
src/main/resources	application.yml và web demo (thư mục static)
src/test/java	Bộ kiểm thử (21 tệp)
native	Mã C++ JNI, CMake và danh sách ký hiệu xuất
scripts	build-native.sh (biên dịch .so) và deploy-board.sh (đẩy lên board)
tools/load-test.mjs	Công cụ thử tải WebSocket (Node 22 trở lên)
.github	Cấu hình CI và Dependabot

PHỤ LỤC C. BIỂU MẪU GHI SỐ ĐO TRÊN BOARD G520

Điền sau khi đo theo quy trình mục 6.6 và 6.5. Mọi ô hiện ghi “Chưa đo”.

Bảng 45. *Thông tin môi trường đo*

Hạng mục	Giá trị
Ngày đo, người đo	Chưa đo
Model board, tổng RAM, nhiệt độ môi trường	Chưa đo
Phiên bản mã (commit)	Chưa đo
Tệp tiếng nói dùng để đo (tên, thời lượng)	Chưa đo
Giá trị stt.pool.max-sessions và stt.threads khi đo	Chưa đo

Bảng 46. *Kết quả thử tải thời gian thực theo số kết nối*

Số kết nối	p50 lagMs	p95 lagMs	p99 lagMs	Bị ngắt hoặc từ chối	RAM (MB)	stt.queue.wait
1	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
2	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
3	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
4	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
5	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
6	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo
7	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo	Chưa đo

Bảng 47. *Kết quả độ chính xác (WER)*

Nhóm tệp	Số từ chuẩn	WER không refiner	WER có refiner
Tiếng Việt, họp thực tế	Chưa đo	Chưa đo	Chưa đo
Tiếng Việt, môi trường ồn	Chưa đo	Chưa đo	Chưa đo
Tiếng Anh	Chưa đo	Chưa đo	Chưa đo

Bảng 48. *Thời gian xử lý một lần end và tệp dài*

Nội dung	Giá trị
Thời gian xử lý một câu 10 giây (một người dùng)	Chưa đo
Thời gian nạp lại mô hình ASR sau refine	Chưa đo
Thời gian xử lý tệp họp 1 giờ (có refiner)	Chưa đo
Ảnh hưởng của một job đang chạy lên độ trễ thời gian thực	Chưa đo